Artificial Intelligence

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

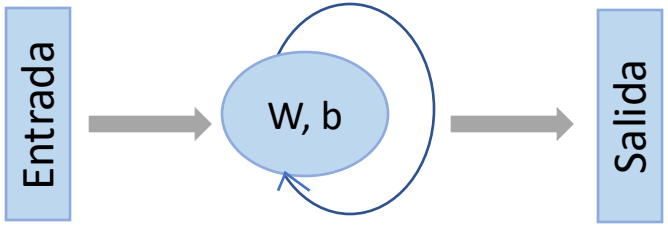
Clase # 14 - PAO 2024 – II

Agenda

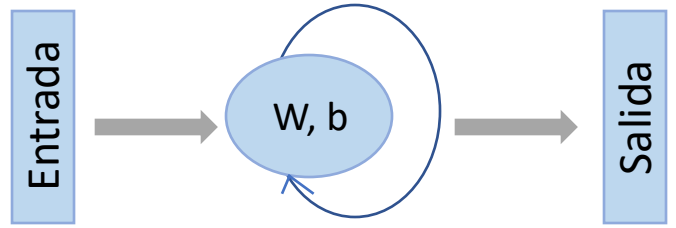
1. Redes neuronales recurrentes
   1.1 Ejemplo procesamiento de lenguaje con RNN

Redes neuronales recurrentes

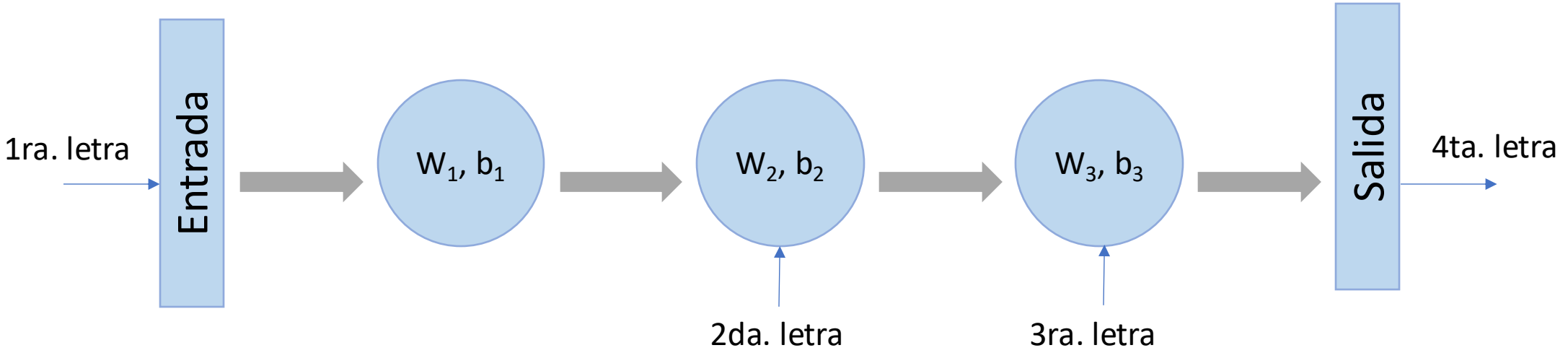
- Es otro tipo de red neuronal que está dominando la solución de problemas en el aprendizaje automático / autónomo y que involucra secuencias de entradas
- Estas redes tienen capas con conexiones en forma de lazos, que agregan retroalimentación y memoria a las capas a lo largo del tiempo
- Esa memoria permite a este tipo de redes aprender y generalizar a través de secuencias de entradas en lugar de patrones individuales



Redes neuronales recurrentes



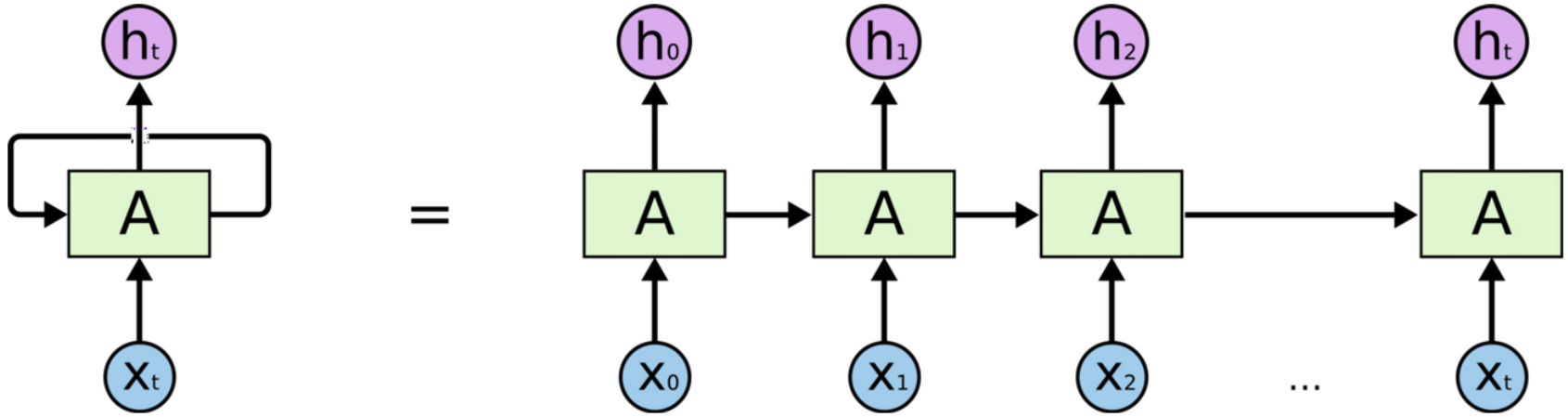
- Cómo funcionan:



- En las RR los pesos y bias de cada capa oculta ya no son independientes
- Para preservar la memoria, es necesario que los pesos y bias prevalezcan en todas las capas

Redes neuronales recurrentes

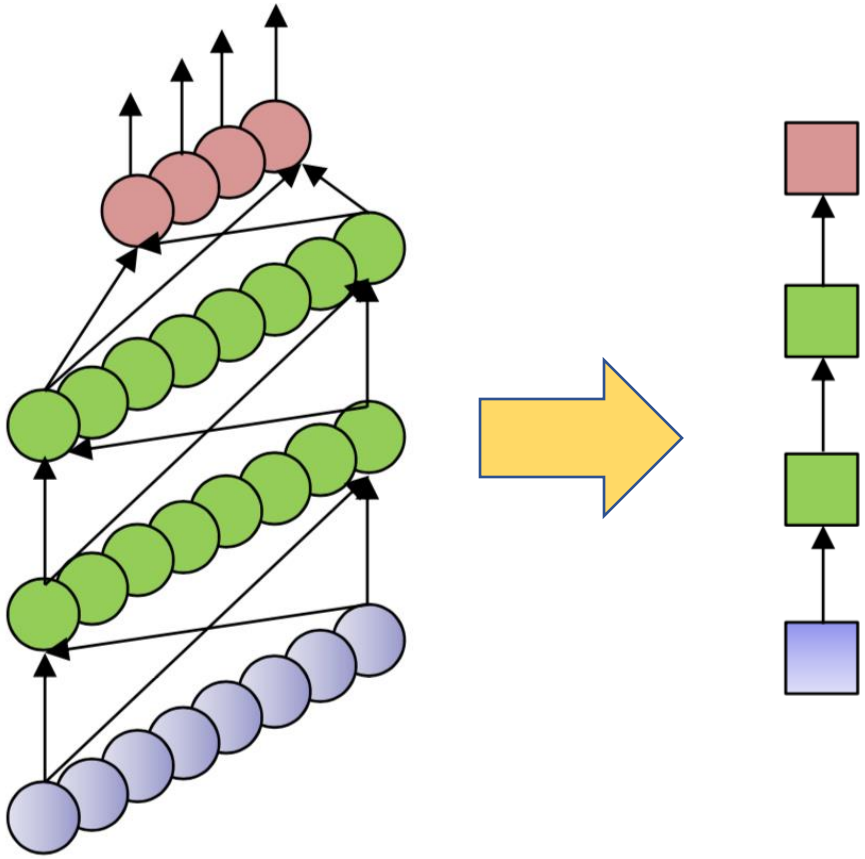
- La RN A recibe una entrada x_t y genera un valor h_t
- El lazo permite que la información se transfiera de un paso al siguiente
- A las RNN podemos considerarlas como una red que contiene múltiples copias de si misma, cada una de las cuales pasa un mensaje a la siguiente



Source: http://colah.github.io/posts/2015-08-Understanding-LSTMs/

Redes neuronales recurrentes

- En realidad algo así:



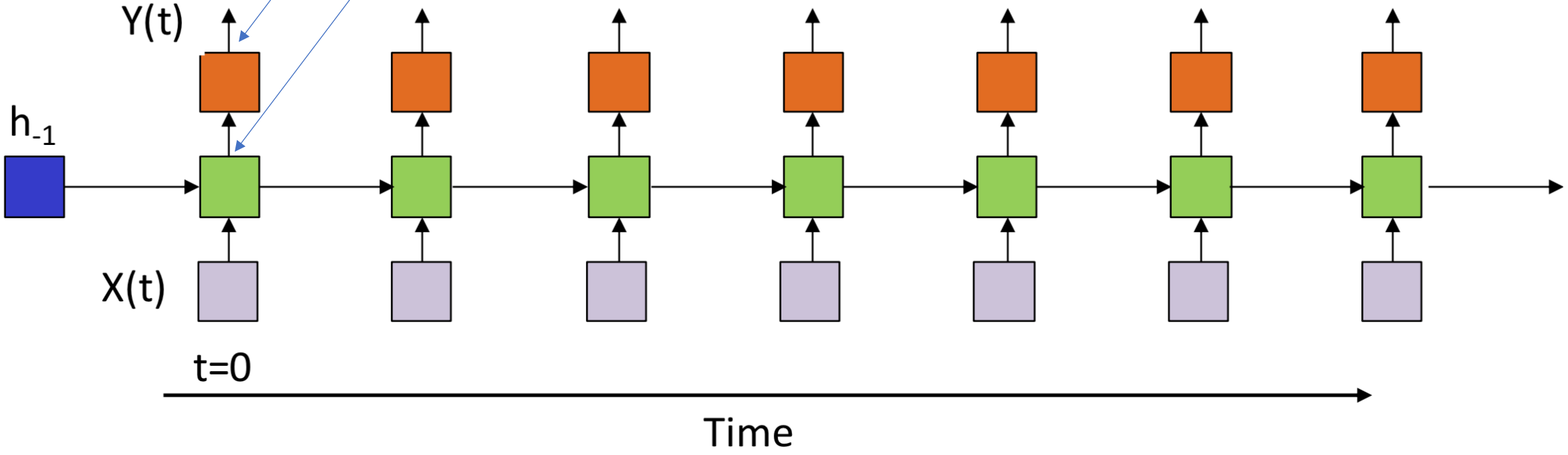
- La entrada en cada momento es un vector
- Cada capa tiene muchas neuronas
  - La capa de salida también puede tener muchas neuronas
- Cada capa la estamos representando con cajas
  - Cada caja en realidad representa una capa entera con muchas unidades

Redes neuronales recurrentes

- El modelo simple *estado-estado*

$$y_t = g(h_t)$$

$$h_t = f(X_t, h_{t-1})$$

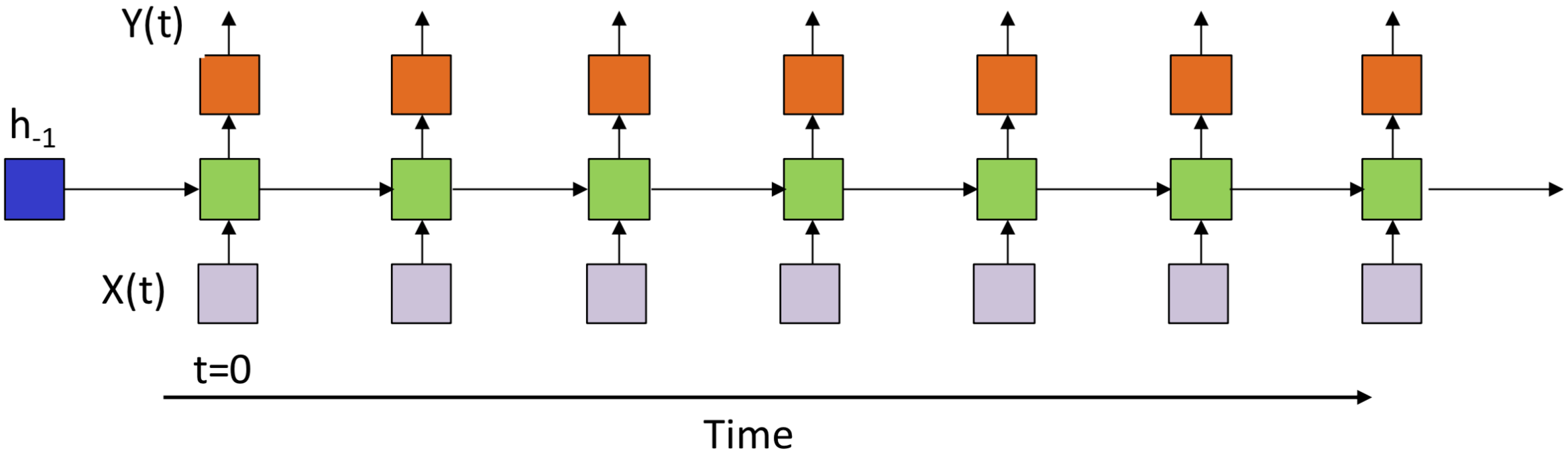


Redes neuronales recurrentes

- El modelo simple *estado-estado*

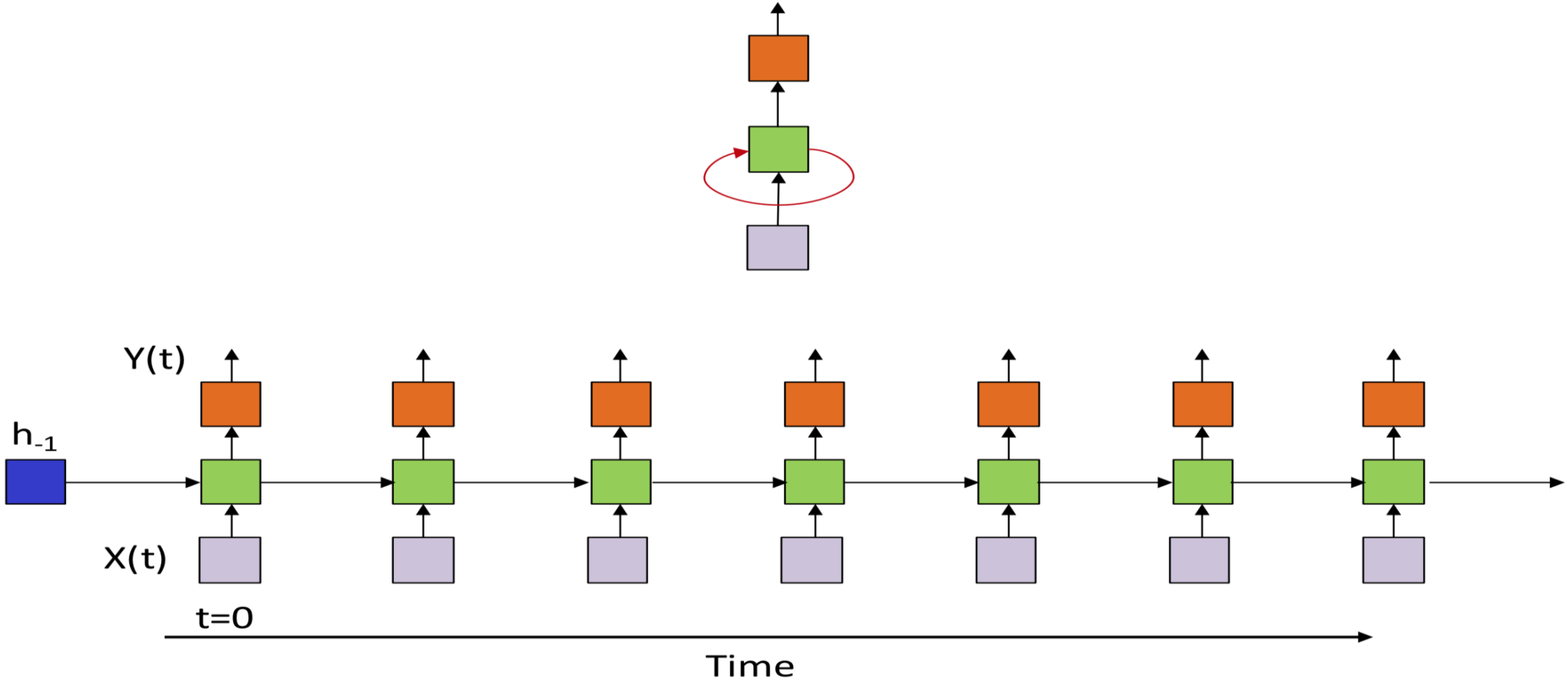
$$h_t = f(X_t, h_{t-1})$$
$$y_t = g(h_t)$$

- h_t, es el estado de la red
- Se requiere la definición inicial de h_{t-1}



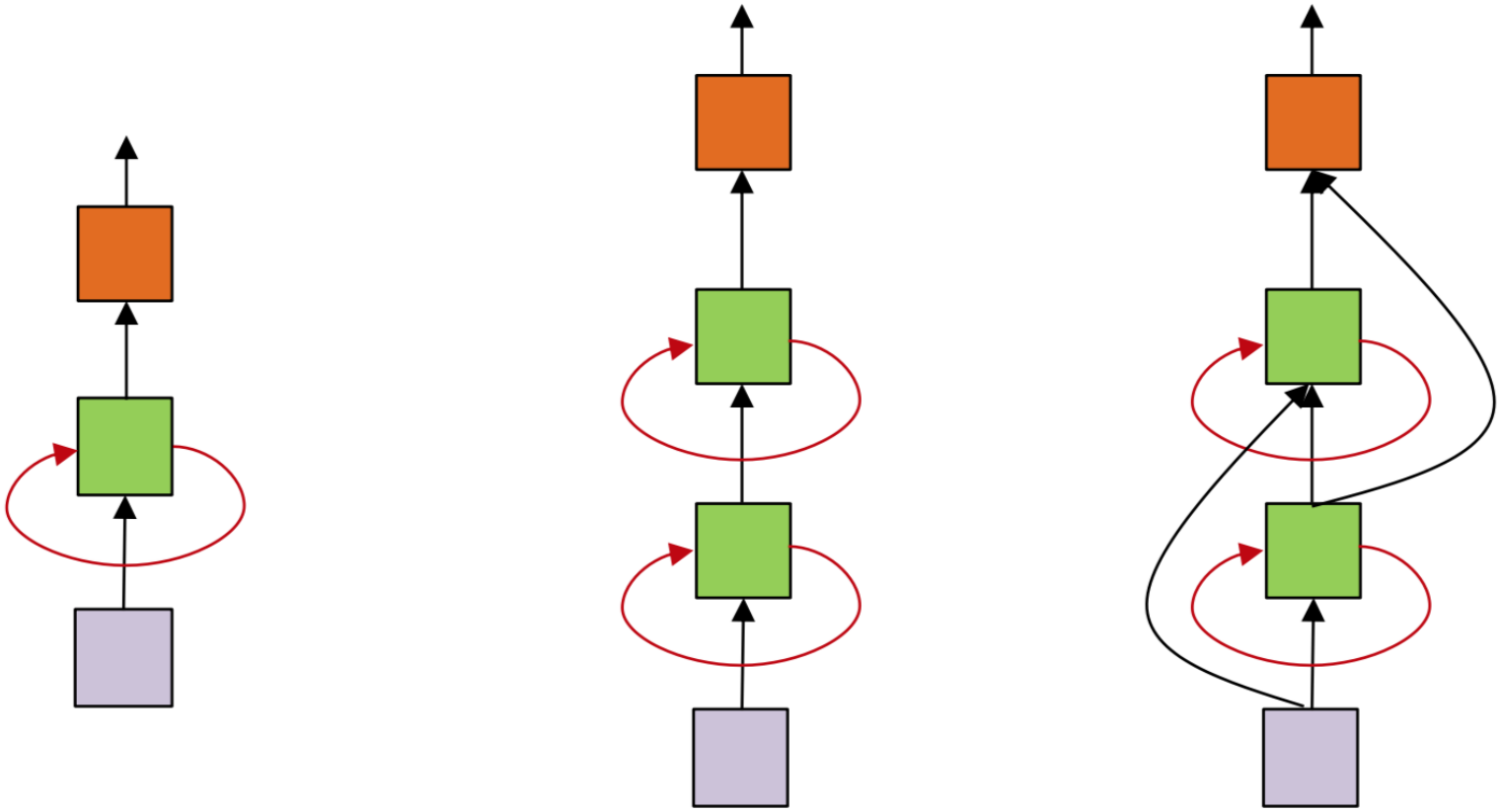
Redes neuronales recurrentes

- Representación simplificada: Una capa escondida



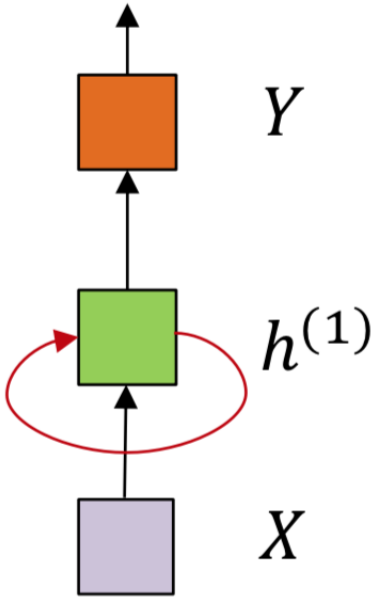
Redes neuronales recurrentes

- Representación simplificada de los modelos
  - Lo lazos implican recurrencia



Redes neuronales recurrentes

- Ecuaciones:



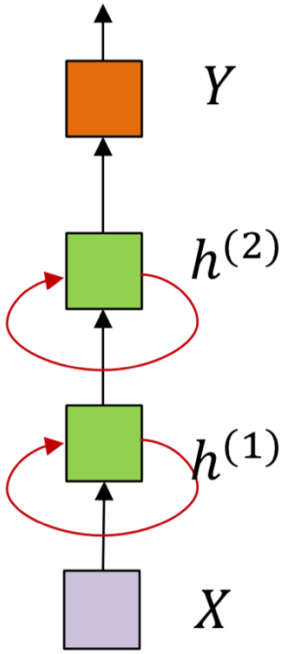
$$Y(t) = f_2\left(\sum_j w_{jk}^{(1)} h_j^{(1)}(t) + b_k^{(1)}, k = 1 \dots M\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(0)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

- Notar que el superscript indica la capa de la red desde donde proviene la entrada
- Asumiendo la función de salida $f_2()$ - Softmax
- La función de activación en los nodos de la capa escondida $f_1()$ generalmente es *tanh()*
- Recordar cada neurona también tiene como entrada un *bias*

Redes neuronales recurrentes

- Ecuaciones:



$$Y(t) = f_3\left(\sum_j w_{jk}^{(2)} h_j^{(2)}(t) + b_k^{(3)}, k = 1 \dots M\right)$$

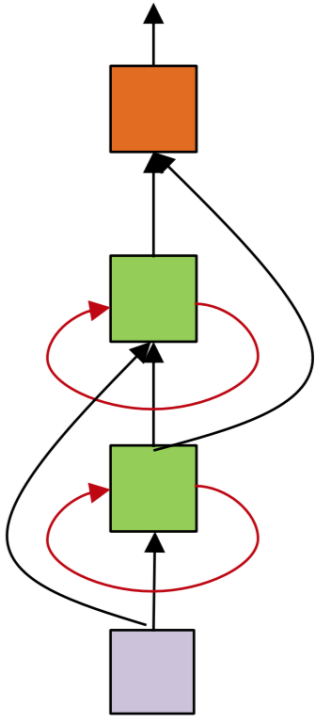
$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(1)} h_j^{(1)}(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(0)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

- Asumiendo la función de salida $f_3()$ - Softmax
- La función de activación en los nodos de la capa escondida $f_k()$ generalmente es *tanh()*
- Recordar cada neurona también tiene como entrada un *bias*

Redes neuronales recurrentes

- Ecuaciones:

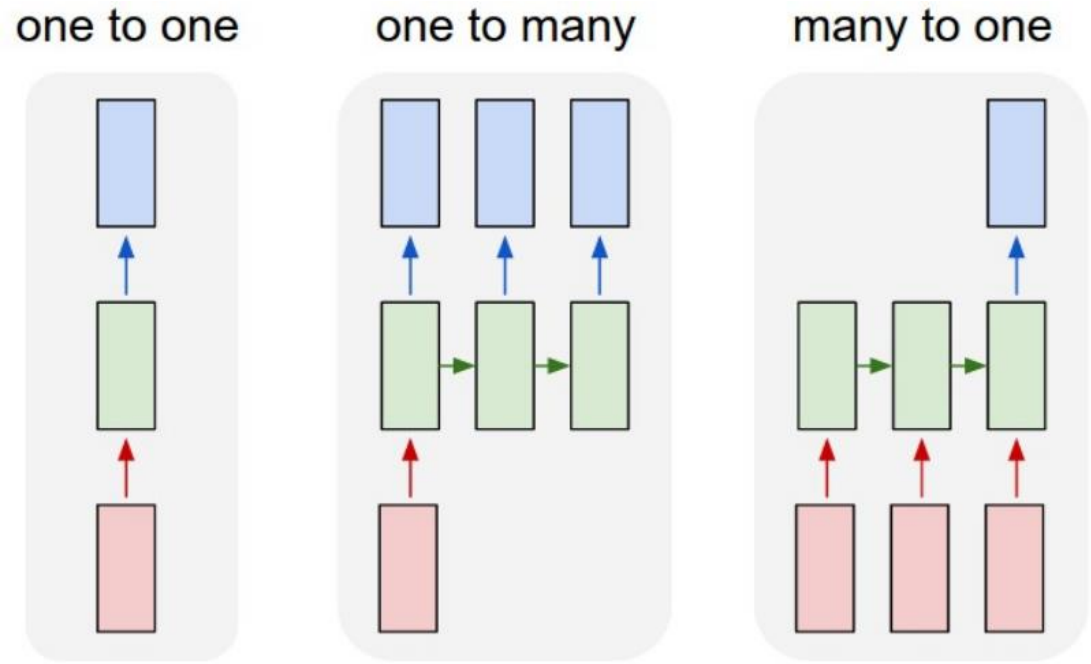


$$Y(t) = f_3\left(\sum_j w_{jk}^{(23)} h_j^{(2)}(t) + \sum_j w_{ji}^{(13)} h_i^{(1)}(t) + b_k^{(3)}, k = 1 \dots M\right)$$

$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(12)} h_j^{(1)}(t) + \sum_j w_{ji}^{(02)} X_j(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

Redes neuronales recurrentes

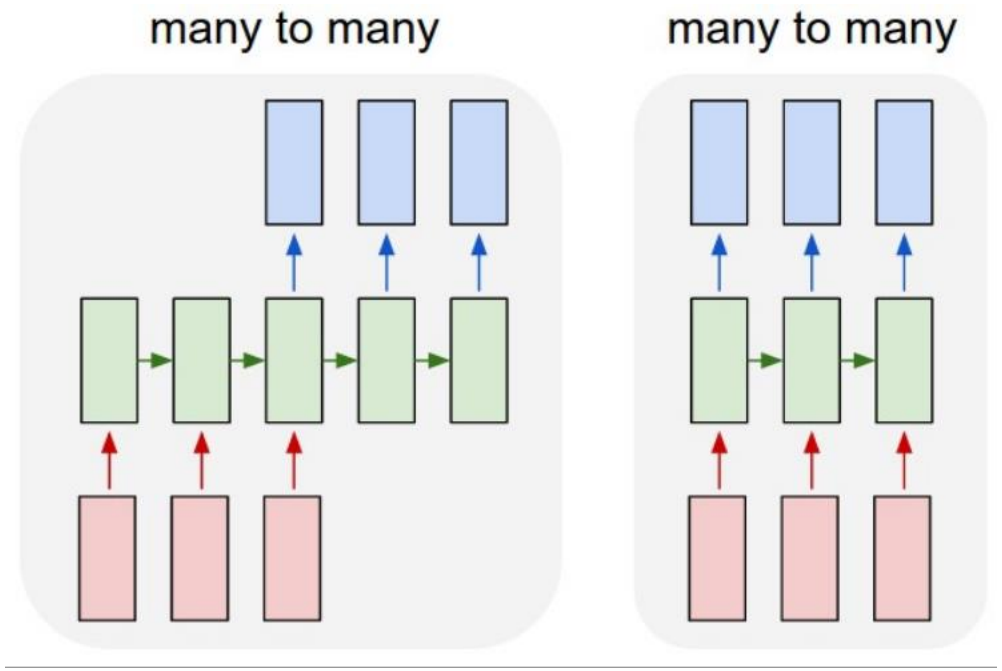


Tipos de RNN:

1. one to one – similar a la MLP convencional
2. one to many – Generación de secuencias: descripción de una imagen (image caption)
3. many to one – Predicción o clasificación basada en secuencias: clasificación de texto, reconocimiento de voz

Fuente: Andrej Karpathy - http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Redes neuronales recurrentes



Tipos de RNN:

4. many to many – Secuencia a secuencia: predicción de la bolsa de valores, predicción de etiquetas, traducción de texto, composición de texto…

Fuente: Andrej Karpathy - http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Redes neuronales recurrentes

Modelado del lenguaje

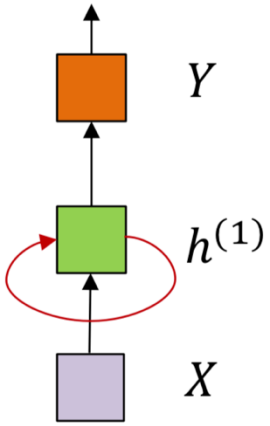
- El modelo de lenguaje se denomina modelo *n-gram*
- Un *n-gram* es un conjunto de *n* palabras consecutivas
- Por ejemplo, en la oración: "Estoy escribiendo un ...", tendríamos los siguientes *n-grams*:
  - unigrams: ”Estoy", "escribiendo", ”un”
  - bigrams: "Estoy escribiendo", "escribiendo un”
  - trigramas: "Estoy escribiendo un”
  - 4-grams: ….

Redes neuronales recurrentes

Modelado del lenguaje

- Las RNN utilizan información secuencial y se denominan recurrentes porque realizan la misma tarea para cada elemento de la secuencia, y la salida depende de los cálculos anteriores
- Idealmente las RNN podrían utilizar la secuencia de información arbitrariamente larga del pasado, pero en la práctica, se limitan a unos pocos pasos en el pasado

Redes neuronales recurrentes



Procedimiento:

1. Ingresamos un caracter a la vez (un time step), es decir X_t se suministra a la red
2. Luego calculamos su estado actual utilizando la combinación de la entrada actual y el estado anterior, es decir, calculamos h_t
3. Este h_t actual se convierte en h_{t-1} para el siguiente paso de tiempo
4. Podemos seguir tantos pasos de tiempo como el problema lo requiera y combinar la información de todos los estados anteriores
5. Una vez que se completan todos los pasos de tiempo, se utiliza el estado actual final para calcular la salida Y_t
6. La salida se compara con la salida deseada y se calcula el error
7. Luego, el error se propaga hacia atrás de la red para actualizar los pesos... Al final la red está entrenada

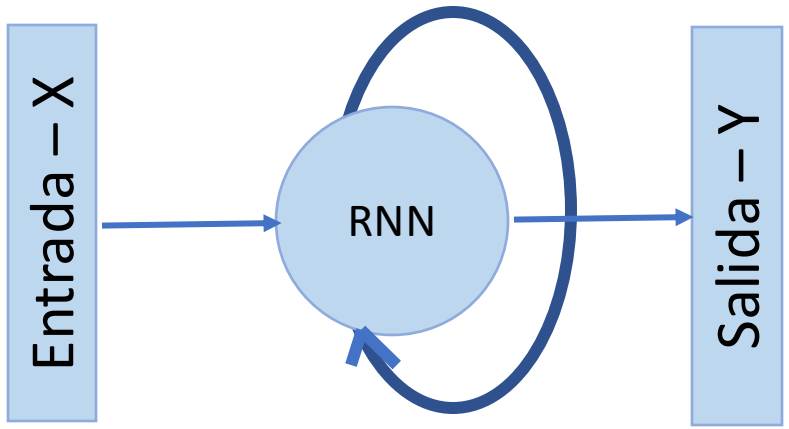
Redes neuronales recurrentes

Modelado del lenguaje

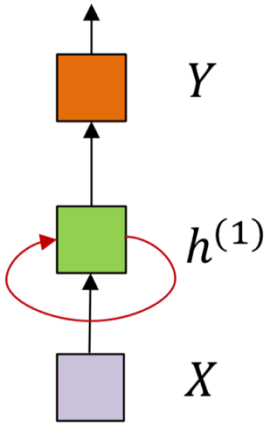
- Las RNN utilizan información secuencial y se denominan recurrentes porque realizan la misma tarea para cada elemento de la secuencia, y la salida depende de los cálculos anteriores
- Idealmente las RNN podrían utilizar la secuencia de información arbitrariamente larga del pasado, pero en la práctica, se limitan a unos pocos pasos en el pasado

Redes neuronales recurrentes

- Por ejemplo: Supongamos una RNN, con la que queremos predecir caracteres
- Supongamos que tenemos la palabra “feliz”; y, luego de ingresar las primeras 4 letras: “f, e, l, i”, queremos que la RNN prediga la siguiente letra: “z”
- En este ejemplo el vocabulario tiene 4 letras: {f, e, l, i}; este vocabulario se denomina “Corpus”, en la práctica, este puede llegar a tener millones de palabras (por ejemplo, la base completa de Wikipedia)
- Veamos cómo esta estructura es utilizada para predecir la quinta letra en la palabra “feliz”



Redes neuronales recurrentes

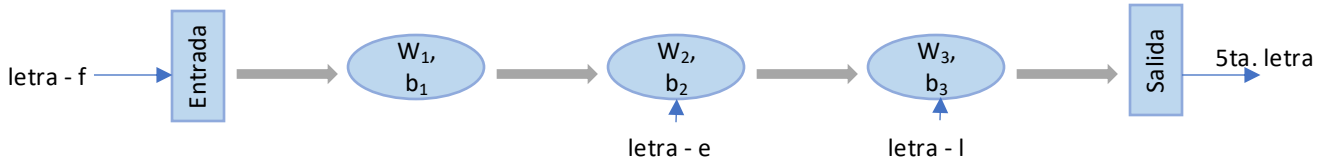


Procedimiento:

1. Ingresamos un caracter a la vez (un time step), es decir X_t se suministra a la red
2. Luego calculamos su estado actual utilizando la combinación de la entrada actual y el estado anterior, es decir, calculamos h_t
3. Este h_t actual se convierte en h_{t-1} para el siguiente paso de tiempo
4. Podemos seguir tantos pasos de tiempo como el problema lo requiera y combinar la información de todos los estados anteriores
5. Una vez que se completan todos los pasos de tiempo, se utiliza el estado actual final para calcular la salida Y_t
6. La salida se compara con la salida deseada y se calcula el error
7. Luego, el error se propaga hacia atrás de la red para actualizar los pesos... Al final la red está entrenada

Redes neuronales recurrentes

- Ingresamos las letras en secuencia:
  - En nuestro ejemplo la letra ”f", la cual no tiene nada que la preceda
  - Luego ingresamos la letra ”e”
  - En el momento que suministramos la letra ”e", se aplica la fórmula de recurrencia a esa letra, y al estado anterior de la capa escondida, en este caso la letra ”f”
  - Este proceso de ingresar cada letra se conoce como pasos de tiempo de la entrada (time steps)
  - Si en el momento t, la entrada es ”e", en el tiempo $t - 1$, la entrada fue ”f"
  - Para obtener el nuevo estado, la fórmula de recurrencia se aplica tanto a “e” como a “f”, así:



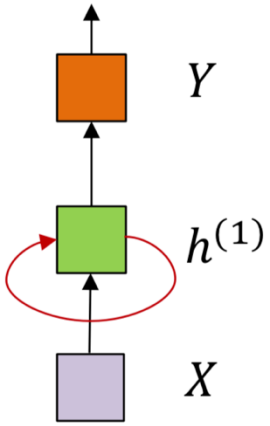
Redes neuronales recurrentes

- Veamos el paso “forward” en nuestro ejemplo:

f		e		l		i
1		0		0		0
0		1		0		0
0		0		1		0
0		0		0		1

- Las entradas están en formato: one-hot-encoded
- El vocabulario completo es: {f, e, l, i}

Redes neuronales recurrentes



- Supongamos una capa con 3 neuronas de ancho
- Entonces necesitamos una matriz W_{xh} de dimensión 3 (neuronas) x 4 (tamaño del corpus)

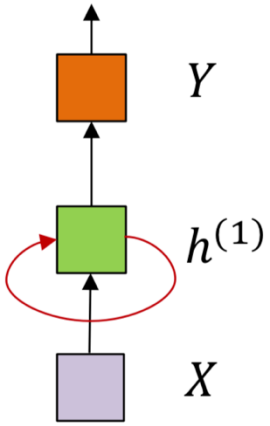
W_{xh}			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

W_{hy}		
0.37168	0.974829459	0.830034886
0.39141	0.282585823	0.659835709
0.64985	0.09821557	0.334287084
0.91266	0.32581642	0.144630018

W_{hh}
0.427043

Bias
0.56700

Redes neuronales recurrentes



- Para la letra “f”, como no tenemos un estado previo, el estado es [0, 0, 0], entonces el estado actual, en el paso 0, es:

$$h_0 = tanh(W_{hh}h_{0-1} + W_{xh}X_0)$$

- En el cálculo $W_{hh}h_{t-1}$, no olvidar el bias: $W_{hh}h_{t-1} + bias$

W_{hh}
0.427043

$\times$

h_{0-1}
0
0
0

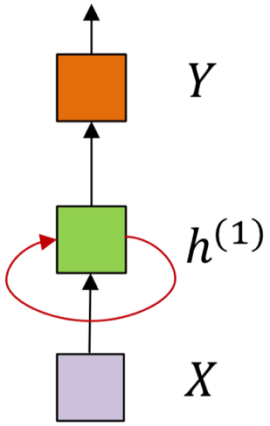
^{T} $+$

Bias
0.56700

$=$

$W_{hh}h_{0-1}$+bias
0.567001
0.567001
0.567001

Redes neuronales recurrentes

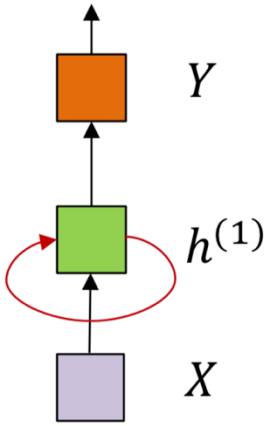


- Entonces: $h_0 = tanh(W_{hh}h_{0-1} + W_{xh}X_0)$, es:

$$h_0 = \tanh\left\{\begin{bmatrix}0.567001\\0.567001\\0.567001\end{bmatrix} + \begin{bmatrix}0.287027359\\0.902874425\\0.537523791\end{bmatrix}\right\}$$

$$h_0 = \begin{bmatrix}0.69316804\\0.89955366\\0.80211840\end{bmatrix}$$

Redes neuronales recurrentes

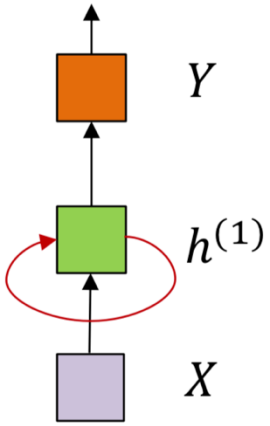


- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$

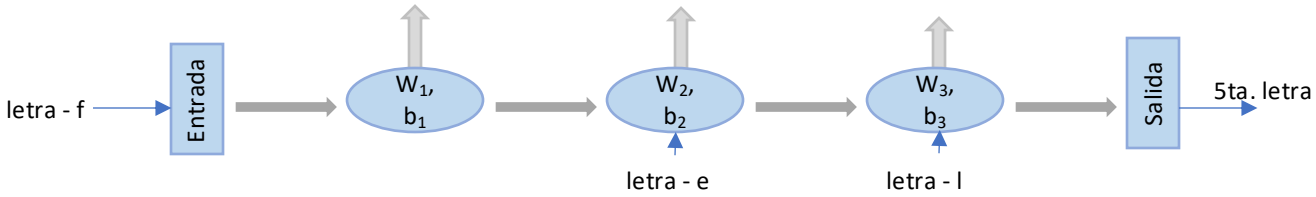
Redes neuronales recurrentes



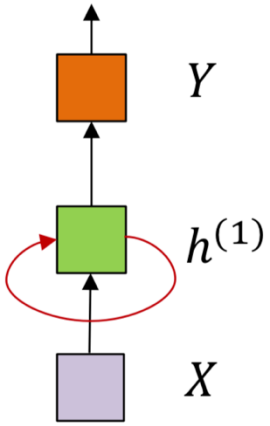
- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$



Redes neuronales recurrentes



- Finalmente, calculamos la probabilidad de las letras en el vocabulario, aplicando la función softmax
- La letra en particular que el sistema recomendaría, en nuestro ejemplo, después de “e”, sería la letra que obtenga la mayor probabilidad, luego de aplicar softmax
- En nuestro ejemplo:

Probabilidades	=	Softmax		Y_1		Probabilidades
				1.90607732		0.419748
				1.13779113		0.194682
				0.95666016		0.162429
				1.27422602		0.223141